

Manual Técnico Detallado

Sistema Polysense

Documentación Generada Automáticamente

16 de diciembre de 2025

Índice

1. Introducción

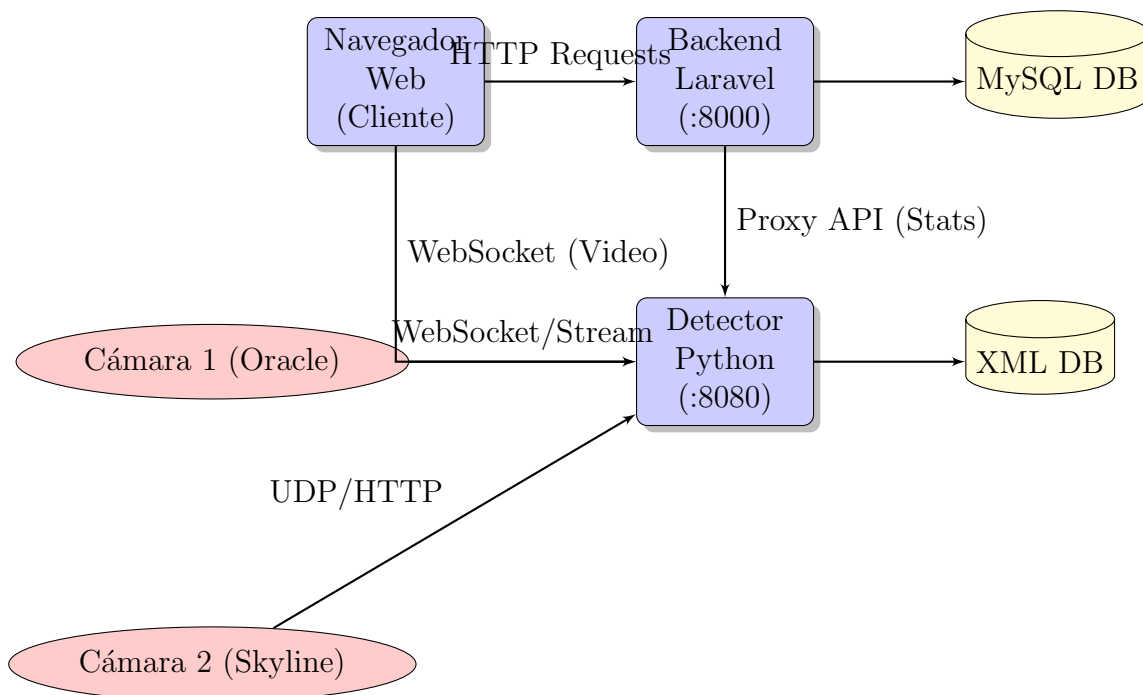
El sistema **Polysense** es una solución híbrida de monitoreo de tráfico vehicular y gestión de cámaras en tiempo real. Combina la robustez de un backend web en **Laravel (PHP)** con la potencia de procesamiento de imágenes e inteligencia artificial de **Python** utilizando **YOLOv8**.

El objetivo principal es detectar, clasificar y contar vehículos de múltiples fuentes de video (Cámaras IP, Streams Web) y presentar estadísticas en tiempo real a través de una interfaz web moderna y responsiva.

2. Arquitectura del Sistema

La arquitectura sigue un patrón de **Microservicios Simplificado** o híbrido, donde el servicio de detección corre independientemente y se comunica con el frontend y backend principal vía HTTP y WebSockets.

2.1. Diagrama de Arquitectura Global



2.2. Flujo de Datos

1. **Ingesta de Video:** El script `vehiculo_detector.py` consume streams de video de múltiples fuentes (configuradas en `CAMERAS_CONFIG`).
2. **Procesamiento IA:** Cada frame es procesado por **YOLOv8** para detectar clases definidas (autos, buses, camiones).
3. **Persistencia:** Las detecciones se guardan en un archivo XML (`vehiculos_db.xml`) para persistencia ligera y rápida sin bloqueo de base de datos relacional.

4. **Transmisión:** Los frames procesados se envían al navegador vía WebSocket en tiempo real.
5. **Gestión Web:** Laravel gestiona la autenticación, roles de usuario, y sirve las vistas que consumen tanto los datos de Laravel (MySQL) como los del detector (API Python).

3. Patrones de Diseño Identificados

3.1. En el Backend (Laravel)

- **MVC (Model-View-Controller):** Estructura fundamental. La lógica de negocio está en Controladores (`app/Http/Controllers`), la presentación en Vistas (`resources/views`), y los datos en Modelos (`app/Models`).
- **Facade Pattern:** Utilizado ampliamente por Laravel (`Auth::check()`, `Route::get()`) y personalizado en el sistema (posiblemente `ReportFacade`).
- **Service Layer:** Se identificaron servicios como `ReportService` y `NotificationService` para desacoplar lógica compleja de los controladores.
- **Dependency Injection:** Inyección de dependencias en constructores y métodos de controladores (ej. `Request $request`).

3.2. En el Detector (Python)

- **Observer / Pub-Sub:** Implementado mediante WebSockets. Los clientes se suscriben a un feed de video y el servidor publica los frames procesados a todos los suscriptores activos.
- **Worker Thread Pattern:** Uso de hilos (`threading.Thread`) para procesar cada cámara de forma concurrente sin bloquear el hilo principal de Flask.
- **Singleton (Implícito):** El modelo YOLO se carga una sola vez y es compartido por los threads.

4. Análisis Detallado de Controladores

4.1. AuthController.php

Gestiona el ciclo de vida de la autenticación de usuarios.

- **Métodos Clave:**
 - `login(Request $request)`: Valida credenciales e inicia sesión con `Auth::attempt`.
 - `register(Request $request)`: Crea nuevos usuarios con rol por defecto 'user'.
 - `logout()`: Invalida la sesión y regenera el token CSRF por seguridad.

4.2. VehicleMonitorController.php

Actúa como un **Proxy** o **Gateway** hacia el microservicio de Python. Evita que el cliente JS tenga que conocer la lógica interna de la API de Python para ciertos datos estadísticos.

- **Variable:** `$detectorUrl = 'http://localhost:8080'` define la dirección del servicio Python.
- **Función `getStats($cameraId)`:**
 1. Recibe el ID de la cámara.
 2. Realiza una petición HTTP interna (`Http::get`) a la API de Python.
 3. Retorna JSON estandarizado al frontend, manejando errores de conexión (`try-catch`) para que la UI no se rompa si Python está caído.

4.3. ReporteController.php

Controlador principal para la visualización y exportación de datos históricos.

- **Persistencia XML:** Lee directamente de `storage/app/vehiculos_db.xml` usando `simplexml_load_string`.
- **Método `obtenerRegistrosFiltrados()`:**
 - Aplica filtros de fecha, tipo de vehículo y cámara sobre el XML.
 - Realiza ordenamiento en memoria (`usort`) por fecha descendente.
- **Exportación CSV:** Genera archivos CSV al vuelo (`php://output`) con cabeceras correctas para descarga directa.

4.4. ModuloTresController.php (Administración)

Panel de control para administradores.

- **Gestión de Automatización:** Permite configurar umbrales de tráfico y alertas de cámaras offline por usuario, interactuando con `AutomationConfig`.
- **Limpieza de XML:** Configura la retención de datos del archivo XML para evitar crecimiento descontrolado, esencial para el mantenimiento a largo plazo.
- **Visibilidad Global:** Acceso a todas las notificaciones y reportes generados en el sistema.

4.5. VoiceCommandController.php

Permite la gestión (CRUD) de comandos de voz para la accesibilidad del sistema.

- **Base de Datos:** Interactúa con la tabla `voice_commands`.
- **Método `createDefaults()`:** Seed de comandos útiles como “Ir a Detección.”, “Exportar Excel”, facilitando la configuración inicial.
- **API REST:** Provee endpoints JSON para que el frontend (JS) sepa qué comandos escuchar.

5. Servicios y Lógica de Negocio

5.1. NotificationService.php

Servicio centralizado para la gestión de alertas.

- **checkAndNotifyHighTraffic()**: Analiza el XML buscando picos de tráfico en los últimos minutos. Implementa lógica de "debounce" para no saturar al usuario con alertas repetidas (ej. 30 mins).
- **checkAndNotifyCameraOffline()**: Verifica si la última detección en el XML supera un umbral de tiempo, indicando posible fallo de cámara.
- **Notificaciones Masivas**: Capacidad de enviar mensajes prioritarios a todos los administradores.

6. Análisis de Vistas y Frontend

6.1. modulo1.blade.php (Dashboard Principal)

Es la vista más compleja, encargada del monitoreo en tiempo real.

- **Integración WebSocket**: Usa JavaScript nativo para conectar a `ws://localhost:8080/ws/stre`
- **Mecanismo de Renderizado**:
 - Recibe **BLOBs** de imágenes (JPEG) por WebSocket.
 - Usa `URL.createObjectURL()` para renderizar eficientemente en un `<canvas>`.
- **Polling de Estadísticas**: Un `setInterval` consume `/api/vehicle-monitor` cada segundo para actualizar contadores y gráficos sin recargar la página.

6.2. modulo2.blade.php (Reportes)

Muestra tablas de datos históricos. Utiliza Blade `@foreach` para iterar sobre los registros filtrados pasados por el `ReporteController`.

7. Rutas (routes/web.php)

El archivo de rutas define la ".API pública" de la aplicación Laravel.

- **Middlewares**:
 - `auth`: Protege rutas que requieren login.
 - `isAdmin`: Protege rutas críticas como el Modulo 3 y la gestión de usuarios.
- **Grupos**: Se utilizan `Route::prefix` y `Route::group` para organizar las rutas por módulo.

8. Componentes Adicionales - Necessary for Function

8.1. Detector Python (vehiculo_detector.py)

El corazón del procesamiento.

- **Flask + Flask-Sock:** Provee tanto la API REST (para stats) como el servidor WebSocket (para video).
- **Playwright:** Se utiliza para extraer URLs de stream complejos (como SkylineWebcams).
- **Multithreading:**
- **Worker Thread:** Captura frames y corre YOLO.
- **Main Thread:** Sirve peticiones HTTP/WS.